

MTH 4600 - Project 8

Volatility Estimation Via Hidden Markov Chain

Martin Bachurin
Anton Gurshumov
John Hemminger

Baruch College

May 22, 2023

This figure plots the hidden markov chain estimate for the annualized time series XOM historical volatility on top of the estimated weighted moving average. We expected that both graphs would look the same because the two models are working off of the same data and are trying to accomplish the same goal. When we plotted the time series, our expectations were confirmed, resulting in nearly identical plots, where one of the only noticeable distinctions is in the period of 2016. The bold line is the HMC estimated sigma that we produced in the code, and the thinner grey line is the plot produced by the EWMA. We can see that the highest historical volatility was seen at around 40%, and then rapidly came back down to around 15%. This is the reason that we can see a distinction. We note that the lowest point of XOM historical volatility of returns was around 10% and had a maximum of 40% over a 4-year time period. The scatter plot also confirmed our expectations, as the relationship between the annualized volatility in percent is clearly linear throughout. The standard normal histogram of returns looks very normal, and aligns with the normal pdf. Lastly, the standardized return time series looks like Gaussian white noise, with the appropriate mean and variance. This aligns with the normal histogram presented.

We all contributed equally to this project, and it was a pleasure working with everyone this semester. I think we will all miss this course. Thank you, Professor Howard, for a great semester.

Listing 1: Code:1

```

int main () {

    // Read in the time series data.
    GetData ();
    // Inititalize obvious values that are always true at t = 0
    sigma[0] = 1.11; // * exp(alpha*0)
    p[0] = 1;

    // sk is an array of all possible values of s evaluated at k.
    for(int j = -1258; j <= 1258; ++j){
        sk[j] = sigma_const * exp(alpha*j);
    }
    // Implement the hidden Markov chain model between here...
    // For each time step, we run the MAP estimation to get kstar
    // This value is used in all estimations of sigma
    for (int t = 1; t < 1259; ++t){
        MAP_kstar(t);
        sigma[t] = sigma_const * exp(alpha*k_star);
        if(t <= 3){
            std::cout << sigma[t] << "\n";
        }
    }
}

Report ();

Pause ();

}

void MAP_kstar(int t){
    // Initialize numerator and denominator for each evaluation.
    // Initialize the percentage to be negative
    double num;
    double den;
    double max = -1;

    // Going forward in state space in steps of two, based on time.

    for(int k = -t; k <= t ; k += 2){
        // reset denominator to zero for each evaluation
        den = 0.0;
        // Using bayesian updating to evaluate the numerator
        // We evaluate the present based on the
        // past 2 states k using the normal pdf function f.
        // On the edge cases, e.g when t = 1 and k = 1
        // the probability of k+1 is 0.

```

```

// In other words when  $k = \pm t$ ,
// the probability of  $k \pm 1$  is evaluated to be 0.
num = f(R[t-1],sk[k-1]) * p[k-1] + f(R[t-1],sk[k+1])*p[k+1];

// Evaluating the denominator as a sum
// This is the probability of the previous data
for(int j = -t+1; j < t ; j += 2){
    den += f(R[t-1],sk[j]) * p[j];
}

den *= 2;
// calculate ptilde
ptilde[k] = num / den;
// set p to ptilde as we move forward.
// ptilde acts as a dummy list.
p[k] = ptilde[k];
}

for(int k = -t; k <= t ; k += 2){
    // update the kstar that maximizes p for a value of k
    if(p[k] > max){
        max = p[k];
        k_star = k;
    }
}
}

////////////////////////////////////
// Normal(0,s^2) density function.
////////////////////////////////////
double f(double r, double s) {

    static double sqrt2pi = 2.506628;

    return exp(-r*r/(2.0*s*s))/(sqrt2pi*s);
}

////////////////////////////////////
// Read in a daily time series of stock price returns  $R[t]$   $1 \leq t \leq 1258$ .
// Allocate array space for the model
////////////////////////////////////
void GetData () {

    int t, n;
    char input[100];
    FILE *fp;

```

```

fp = fopen ("XOM5YrsDaily.txt", "r");

// Read in the file description.
fgets (input, 99, fp);

// Allocate memory for the data, initialized to 0.
R      = List (1300);
R2     = List (1300);
sigma  = List (1300);
EWMA   = List (1300);

// Allocate necessary memory for the HMC model.
s = AllocateMemory (); // You may now refer to s[-1258] through s[1258].
p = AllocateMemory (); // Same comment about p, ptilde, and g.
ptilde = AllocateMemory (); // The magic of pointer arithmetic!
sk = AllocateMemory ();      // Can't do that in Python.

// Now read in the XOM return data (in percent).
for (t = 0; t < 1258; t++) {
    fgets (input, 99, fp);
    sscanf (input, "%lf", R+t);
    R2[t] = R[t] * R[t];
}
fclose (fp);

// Now read in the EWMA data (for purposes of the scatter plot).
fp = fopen ("EWMA.txt", "r");
for (t = 51; t <= 1258; t++) {
    fgets (input, 99, fp);
    sscanf (input, "%d_%%lf", &n, EWMA+t);
}
fclose (fp);

return;
}

////////////////////////////////////
// Generate some output files.
////////////////////////////////////
void Report () {

    int t;
    double annualizedVol, Z;
    FILE *fp1, *fp2;

    // First create a time series of HMC estimated annualized volatility,

```

```

//      and standardized XOM return.

// Create scatter plot and volatility time series data files.
fp1 = fopen ("ScatterData.txt", "w");
fp2  = fopen ("HMC.txt", "w");

// Here "t" can include 1258.
for (t = 51; t <= 1258; t++) {
    annualizedVol = sqrt(252.0) * sigma[t];
    fprintf (fp1, "%6.2f_ %6.2f\n", EWMA[t], annualizedVol);
    fprintf (fp2, "%4d_ %10.2f\n", t, annualizedVol);
}
fclose (fp1);
fclose (fp2);

// Report standardized return information. Here "t" cannot include 1258.
fp1 = fopen ("StandardizedXOM.txt", "w");
for (t = 51; t < 1258; t++) {

    // Standardize the data.
    Z = PsiInv(MTUniform(0));
    //Z = R[t] / sigma[t];
    // !!!!! Get rid of this line ,
    // and un-comment this line.

    // Add the standardized return to the histogram with 20 buckets.
    NormalHistogram (Z, 20, 0);

    // Now put it into the time series file
    fprintf (fp1, "%4d_ %6.3f\n", t, Z);

}
fclose (fp1);

// Create histogram TeX file.
NormalHistogram (0, 20, 1);

return ;

}

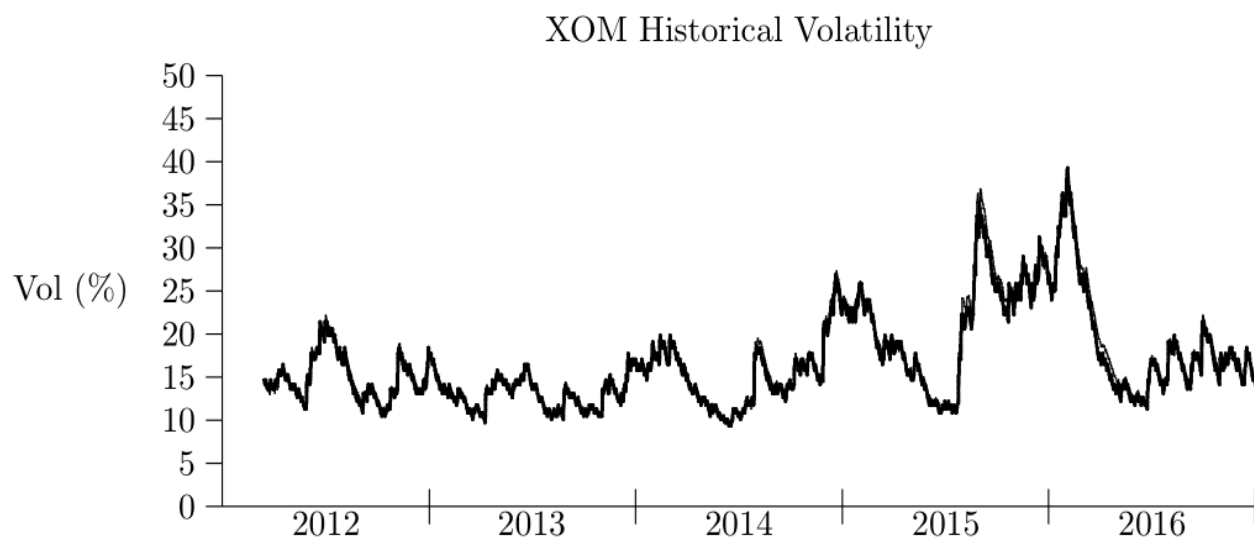
/////////////////////////////////////////////////////////////////
// Allocate memory for an array with indices from -1300 to +1300.
// This is a little more than needed.
/////////////////////////////////////////////////////////////////
double *AllocateMemory () {

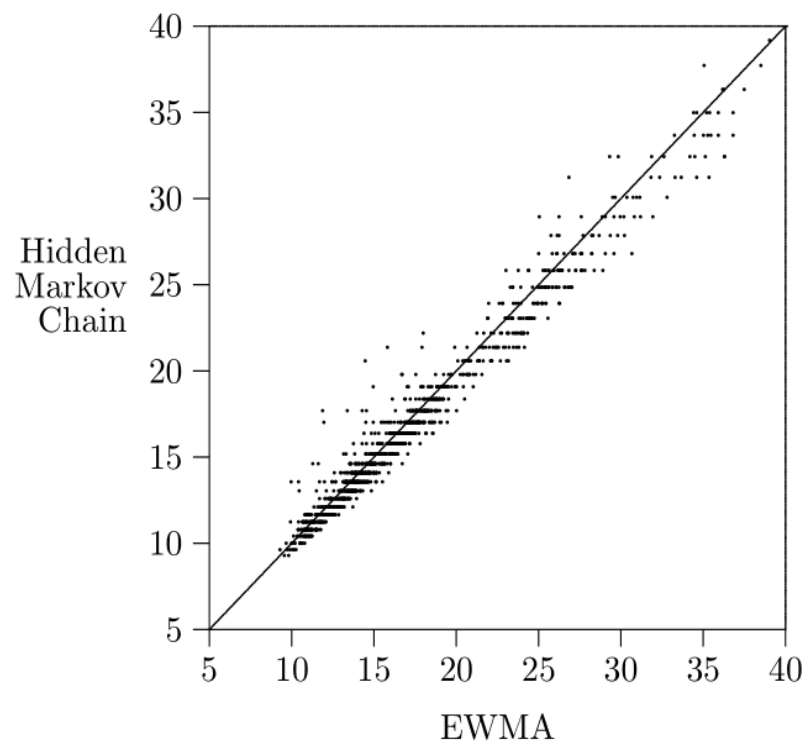
    double *x;

    x = (double *) calloc (2*1300 + 1, sizeof (double));

```

```
x += 1300;  
return x;  
}
```





Standard Normal Histogram

